

Delivery Resource Planning System

Design Document (updated to match the current implementation)

Terminology & Formulae

Core entities

- Order: order_id, distance_km, weight_kg, priority (Express/Standard), deadline_hr, revenue (calculated), status (Pending/Assigned/Rejected), reject_reason, assigned_vehicle_id.
- Vehicle: vehicle_id, type (Bike/Van/Truck), max_capacity_kg (built in from type), remaining_capacity, assigned_orders. Methods: rate_for(priority) returns the built-in Rs/km; cost_of(order) returns order distance x rate_for(order priority).
- Personnel: personnel_id, name.

Built-in constants

Constant	Value
CAPACITY_BY_TYPE (kg)	Bike 15, Van 65, Truck 200
SPEED_BY_TYPE (km/h)	Bike 30, Van 25, Truck 20
RATE_BY_PRIORITY_AND_TYPE (Rs/km)	Standard: Bike 30, Van 55, Truck 70. Express: Bike 55, Van 80, Truck 95
REVENUE_PER_KG	Rs20 per kg
MIN_REVENUE	Rs100
MIN_DISTANCE_KM	0.2 km (200 m)

Vehicle class for an order

vehicle_class_for_weight(weight) returns the smallest vehicle type whose capacity is at least the weight (Bike up to 15 kg, Van up to 65 kg, Truck up to 200 kg), or None if the weight is over 200 kg. It is used only to price an order at entry.

Revenue

Revenue (Rs) = weight_kg x 20 + distance_km x RATE_BY_PRIORITY_AND_TYPE[priority][vehicle class]. It is calculated once, when the order is added, and never entered by the user.

Delivery time

Delivery Time (hr) = Order distance (km) / Vehicle average speed (km/h). An order is time-feasible on a vehicle if Delivery Time <= order deadline_hr.

Trip cost

Trip Cost (Rs) = sum over every order assigned to the vehicle of (order distance_km x vehicle.rate_for(order priority)). The rate depends on the vehicle's type and on each order's priority, so a vehicle carrying a mix of Express and Standard orders is charged at different rates per order.

Profit

Net Profit (Rs) = Total Revenue - Total Trip Cost, summed across all vehicles used in the plan. Profit per kg (used only for ranking) = Order revenue / Order weight_kg.

Order ranking key

Pending orders are sorted by, in order of importance:

1. Priority rank - Express before Standard.
2. Revenue - higher first (the order that pays more is accepted first).
3. Profit per kg - higher first.
4. Deadline urgency - smaller deadline_hr first.
5. Entry order - whichever order was entered first wins any remaining tie.

Vehicle-personnel pairing rule

Vehicles and personnel are each sorted by ID and paired one-to-one in that order. Any vehicle or personnel left over once the shorter list is exhausted is marked idle.

Vehicle selection rule

For each order, every paired vehicle is checked for capacity and time feasibility. Among the feasible vehicles, the one with the lowest cost_of(order) wins. If costs are exactly equal, the vehicle that comes first in the pair list wins, which means the pair with the lowest personnel ID (the first delivery person in the list).

Capacity feasibility

An order is capacity-feasible on a vehicle if $\text{order.weight_kg} \leq \text{the vehicle's remaining capacity}$ (max_capacity_kg minus the weight already assigned).

Input validation and acceptance rules

- distance_km, weight_kg, deadline_hr: must be numeric and strictly greater than 0.
- Order ID / Vehicle ID / Personnel ID: must not already exist among stored records.
- priority: must be 1 (Express) or 2 (Standard). type: must be 1 (Bike), 2 (Van), or 3 (Truck).
- Order refused (not stored) if $\text{distance_km} < 0.2$.
- Order refused (not stored) if $\text{weight_kg} > 200$ (no vehicle class).
- Order refused (not stored) if calculated revenue < 100 .
- Capacity, per-km rates and revenue are never entered, so they need no validation.

Program Flow

1. Display the main menu to the user.
2. Get the user's menu choice (Add Order, View All, Add Vehicle, Add Personnel, Generate Plan, View Last Plan, Exit).
3. If Add Order, Add Vehicle, or Add Personnel is chosen, collect and validate the relevant fields. For Add Order, also calculate revenue and apply the acceptance rules; store the record only if accepted. Return to step 1.
4. If View All is chosen, display all currently stored orders, vehicles, and personnel, and return to step 1.

5. If Generate Daily Delivery Plan is chosen, pair vehicles with personnel, rank pending orders, allocate orders to the cheapest feasible vehicle within capacity and deadline limits, and re-check route-level profitability.
6. Display the generated plan: assigned orders per vehicle, rejected orders with reasons, idle vehicles/personnel, and totals.
7. If View Last Plan / Report is chosen, redisplay the most recently generated plan, or print 'No plan generated yet' if none exists.
8. Handle edge cases (zero orders/vehicles/personnel, duplicate IDs, boundary capacity/deadline values, refused orders, unprofitable orders or routes, ranking ties, cost ties).
9. If Exit is chosen, print a goodbye message and end; otherwise return to step 1.

Function Signatures

Typed header line	What it's for
def vehicle_class_for_weight(weight_kg) -> str None:	Returns the smallest vehicle type that can carry the weight, or None if over 200 kg.
def calculate_revenue(distance_km, weight_kg, priority) -> float:	Returns weight x Rs20 + distance x per-km rate for the order's priority and vehicle class.
def add_order(orders: list) -> Order None:	Prompts for and validates order fields, calculates revenue, applies the acceptance rules. Appends and returns the new Order, or returns None if the order was refused.
def add_vehicle(vehicles: list) -> Vehicle:	Prompts for ID and type, creates a Vehicle with built-in capacity, appends and returns it.
def add_personnel(personnel: list) -> Personnel:	Prompts for ID and name, appends and returns the new Personnel.
def view_all(orders, vehicles, personnel) -> None:	Prints all stored orders (with revenue and status), vehicles (capacity and rates), and personnel.
def pair_vehicles_personnel(vehicles, personnel) -> tuple:	Returns (pairs, idle_vehicles, idle_personnel); pairs are (vehicle, driver) in ID order.
def rank_orders(orders: list) -> list:	Returns pending orders sorted by priority, revenue, profit per kg, deadline, and entry order.
def delivery_time(order, vehicle) -> float:	Returns hours needed to deliver the order at the vehicle's average speed.
def trip_cost(vehicle, assigned_orders: list) -> float:	Returns the total Rs cost: sum of each order's distance x the vehicle's rate for that order's priority.
def allocate_orders(ranked_orders, paired_vehicles) -> dict:	Assigns each ranked order to the cheapest feasible vehicle (ties to the first pair), rejects the rest. Returns {'pairs', 'rejected'}.
def check_route_profitability(plan: dict) -> dict:	Drops the worst-margin orders from each route until the route is profitable, rejecting each with a reason. Returns the plan.
def generate_plan(orders, vehicles, personnel) -> dict:	Runs the whole planning pipeline, handles the zero-orders/vehicles/personnel cases, and returns the plan dict used by build_report.
def build_report(plan: dict) -> str:	Formats a plan into the DELIVERY PLAN - SUMMARY text block.
def read_positive_number(prompt: str) -> float:	Loops until the user enters a number greater than 0, and returns it.

Typed header line	What it's for
<code>def read_choice(prompt: str, valid: list) -> str:</code>	Loops until the user enters one of the valid options, and returns it.
<code>def read_unique_id(prompt: str, existing: list) -> str:</code>	Loops until the user enters an ID not already in existing, and returns it.
<code>def main() -> None:</code>	Runs the menu loop: displays options, dispatches to the right function, repeats until Exit.

Function-Level Algorithm

vehicle_class_for_weight(weight_kg)

1. For each type in order Bike, Van, Truck: if weight_kg <= that type's capacity, return the type.
2. If none fits, return None.

calculate_revenue(distance_km, weight_kg, priority)

1. Find the vehicle class with vehicle_class_for_weight.
2. Per-kg part = weight_kg x 20.
3. Per-km part = distance_km x RATE_BY_PRIORITY_AND_TYPE[priority][class].
4. Return per-kg part + per-km part.

add_order(orders)

1. Prompt for and validate Order ID via read_unique_id.
2. Prompt for distance via read_positive_number. If distance < 0.2 km, print a message and return None.
3. Prompt for weight via read_positive_number. If vehicle_class_for_weight is None, print a message and return None.
4. Prompt for priority via read_choice (Express/Standard).
5. Prompt for deadline via read_positive_number.
6. Calculate revenue with calculate_revenue. If revenue < 100, print a message and return None.
7. Create a new Order with status 'Pending', append it to orders, print the calculated revenue, and return the Order.

add_vehicle(vehicles)

1. Prompt for and validate Vehicle ID via read_unique_id.
2. Prompt for and validate type via read_choice (Bike/Van/Truck).
3. Create a new Vehicle (capacity comes from the type), append it to vehicles, and return it.

add_personnel(personnel)

1. Prompt for and validate Personnel ID via read_unique_id.
2. Prompt for a name.
3. Create a new Personnel, append it to personnel, and return it.

view_all(orders, vehicles, personnel)

1. Print all orders with their fields (including calculated revenue and status), or 'No orders added yet'.

2. Print all vehicles with type, capacity, and Standard/Express per-km rates, or 'No vehicles added yet'.
3. Print all personnel, or 'No personnel added yet'.

pair_vehicles_personnel(vehicles, personnel)

1. Sort vehicles and personnel by ID.
2. Pair the i-th vehicle with the i-th personnel until either list runs out.
3. Leftover vehicles are idle ('no personnel available'); leftover personnel are idle ('no vehicle available').
4. Return the pairs plus the two idle lists.

rank_orders(orders)

1. Keep only orders with status 'Pending'.
2. Sort by (priority rank, -revenue, -profit_per_kg, deadline_hr, entry index).
3. Return the sorted list.

delivery_time(order, vehicle)

1. Look up the vehicle's average speed from its type.
2. Divide order.distance_km by the speed and return the hours.

trip_cost(vehicle, assigned_orders)

1. For each order, compute $\text{vehicle.cost_of}(\text{order}) = \text{distance_km} \times \text{vehicle.rate_for}(\text{order.priority})$.
2. Return the sum.

allocate_orders(ranked_orders, paired_vehicles)

1. Reset every paired vehicle's remaining capacity and assigned orders.
2. For each order in ranked_orders, in order: build the list of paired vehicles where $\text{weight} \leq \text{remaining capacity}$ and $\text{delivery_time} \leq \text{deadline}$, each with its $\text{cost_of}(\text{order})$ and pair index.
3. If the list is not empty, choose the entry with the lowest (cost, pair index). Assign the order to that vehicle, reduce its remaining capacity, set status 'Assigned'.
4. If the list is empty, mark the order 'Rejected' with reason 'no vehicle had capacity/time'.
5. Return {'pairs': paired_vehicles, 'rejected': rejected}.

check_route_profitability(plan)

1. For each vehicle in plan['pairs'] that has assigned orders:
2. Calculate trip_cost and the total revenue of its assigned orders.
3. While trip cost exceeds revenue and the vehicle still has orders, remove the order with the lowest $(\text{revenue} - \text{vehicle.cost_of}(\text{order}))$, mark it 'Rejected' with reason 'unprofitable at route level', add it to plan['rejected'], and recalculate.
4. Return the plan. (generate_plan marks vehicles that end up empty as 'no profitable orders'.)

generate_plan(orders, vehicles, personnel)

1. If there are no orders, return a plan with the message 'No orders to plan'.
2. If there are no vehicles, reject every pending order with reason 'no vehicles available' and return.

3. If there are no personnel, reject every pending order with reason 'no personnel available' and return (all vehicles idle).
4. Otherwise pair vehicles with personnel, rank the orders, and run `allocate_orders`.
5. Remember which paired vehicles are empty after allocation, then run `check_route_profitability`.
6. For each paired vehicle that ends empty, set its idle reason: 'no compatible orders' if it was empty after allocation, otherwise 'no profitable orders'.
7. Attach the idle vehicle and personnel lists and the orders to the plan, and return it.

build_report(plan)

1. If the plan holds the 'No orders to plan' message, return it.
2. For each paired vehicle with orders, print its ID, type, driver, each assigned order, total load versus capacity, and trip cost. For an empty one, print IDLE with the reason.
3. Print vehicles idle for lack of personnel and personnel idle for lack of a vehicle.
4. Print the rejected orders with their reasons (or 'None').
5. Print total revenue, total cost, net profit, and orders completed versus total orders.
6. Return the formatted string.

read_positive_number(prompt)

1. Ask the user for input using prompt.
2. If the input is not a number or is less than or equal to 0, print 'Invalid value, must be greater than 0.' and ask again.
3. Return the float.

read_choice(prompt, valid)

1. Ask the user for input using prompt.
2. If the input is not one of valid, print 'Invalid option.' and ask again.
3. Return the chosen value.

read_unique_id(prompt, existing)

1. Ask the user for input using prompt.
2. If the input matches an ID already in existing, print 'ID already exists.' and ask again.
3. Return the ID.

main()

1. Print the main menu.
2. Ask the user to choose an option (1-7).
3. If 1, call `add_order`. If 2, call `view_all`. If 3, call `add_vehicle`. If 4, call `add_personnel`. If 5, call `generate_plan`, then `build_report`, store the plan as the last plan, and print the report. If 6, call `build_report` on the last plan, or print 'No plan generated yet'. If 7, print 'Goodbye!' and end.
4. After any option except Exit, return to step 1.